

Lecture 2

Evan Drake

- ◆ Why pseudocode?
- ◆ Introduction to Variables
- ◆ Introduction to Data Types

Why pseudocode

◆ Ideas?



Why pseudocode?

- ◆ Closer to math
- ◆ Easier to mockup
- ◆ Universal language

Bindings

$$\underbrace{A}_{\text{Name}} \leftarrow \underbrace{B}_{\text{Value}}$$

The **Name** "A" will now represent the **Value** "B"

We bind the name A to the value B

More Bindings

$$\underbrace{A}_{\text{Name}} \leftarrow \underbrace{B}_{\text{Value}}$$
$$x \leftarrow 0$$
$$\textit{squid} \leftarrow 10,000,000,000$$

“A mathematician is a device for turning coffee
into theorems.”

–Paul Erdős

LHS of Bindings

$$\textcircled{A} \leftarrow B$$

Names are simple *words*

Names act the same way as a variable in algebra

RHS of Bindings

$$A \leftarrow \textcircled{B}$$

Recall that A represents the value of B

B is an expression

What is an Expression?

$$x + (\underbrace{6} - \underbrace{7})$$

Constants: symbols directly representing values

What is an Expression?

$$x \underbrace{+}_{\text{operator}} (\underbrace{6}_{\text{operand}} \underbrace{-}_{\text{operator}} \underbrace{7}_{\text{operand}})$$

Operators: functions with defined behavior

What is an Expression?

$$\underbrace{x} + (6 - 7)$$

Bindings

What is an Expression?

$$x + (6 - 7)$$

Expressions can be evaluated (solved)

$$x + (-1)$$

Key feature...

More Expressions

$$17 + (6 - 7)$$

$$18 + 2 + 10$$

$$2 + (100 - 10) - (4 * 16/2)$$

Bindings

$$\underbrace{A}_{\text{Name}} \leftarrow \underbrace{B}_{\text{Expression}}$$

Here we will change the RHS to an expression

Statements

- ◆ Bindings | Variables - synonyms in our case
- ◆ LHS $\rightarrow$ Name
- ◆ RHS $\rightarrow$ Expression
- ◆ Operator ($\leftarrow$)

“To be a good programmer, you need to understand the principles of generic programming. To understand the principles of generic programming, you need to understand abstraction. To understand abstraction, you need to understand the mathematics on which it's based.”

—Alexander A Stepanov

Data Types



Why Data Types?

- ◆ Ease of use
- ◆ Performance
- ◆ Logical legality

Math Reasons

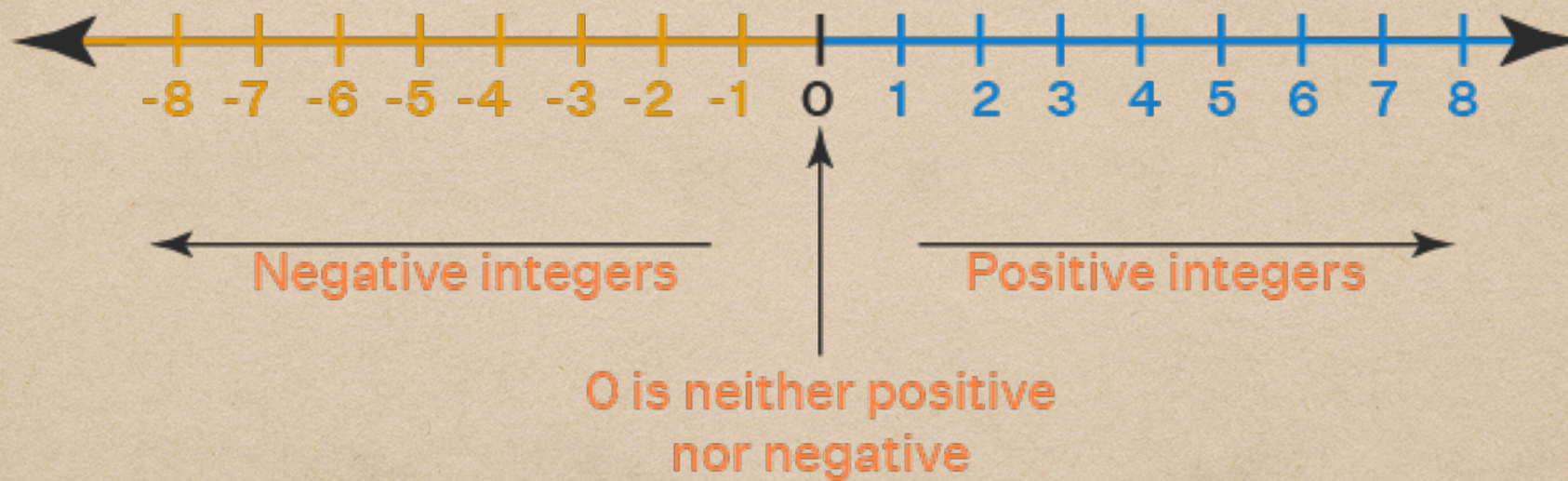
$z \in \mathbb{Z}$ Integers

$r \in \mathbb{R}$ Real numbers

Recall

 $\mathbb{Z}$

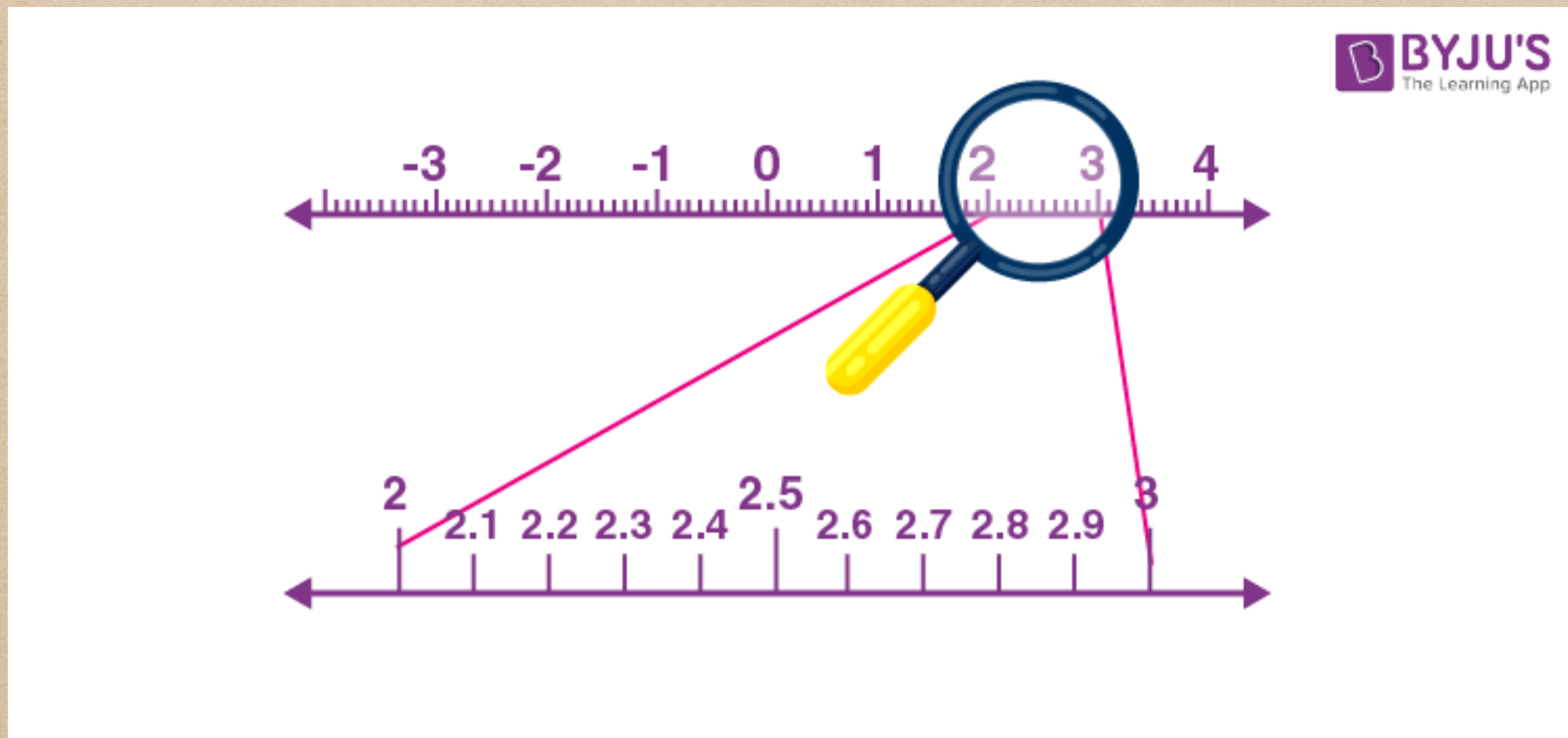
Integers on a Number Line



Recall

Real Numbers

$\mathbb{R}$



Math Reasons

$z \in \mathbb{Z}$ Integers

$r \in \mathbb{R}$ Real numbers

$z + 1$

Logically sound

$r + 1$

$z + \sqrt{2}$

Logically unsound

$r + \sqrt{2}$

Math Reasons

$z \in \mathbb{Z}$ Integers

$r \in \mathbb{R}$ Real numbers

$+: \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$

$z \oplus \sqrt{2}$

Logically unsound

$r \oplus \sqrt{2}$

$+: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$



“Computer science is no more about computers
than astronomy is about telescopes.”

— Edsger Dijkstra

Simple Data Types

- ◆ Numbers: $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{C}$
- ◆ Characters
- ◆ Booleans
- ◆ Collections

Characters

Single character or symbol

Char : [ABC....]

Booleans

B : True | False

Booleans can either be True (1) or False(0)

Collections

Collections : $[(\mathbb{Z} \mid \text{Char} \mid \text{Bool} \dots)]$

Collection of the other types

Homogeneous in most cases